

Aula 09 — Fechamento do pedido

Como evoluir `Pedido` para proteger uma nova regra sem refazer as responsabilidades que já funcionam?

O pedido já calcula o total

```
public void adicionarItem(Produto produto, int quantidade) {  
    ItemPedido item = new ItemPedido(produto, quantidade);  
    itens.add(item);  
}  
  
public double calcularTotal() {  
    double total = 0.0;  
    for (ItemPedido item : itens) {  
        total += item.calcularSubtotal();  
    }  
    return total;  
}
```

A Versão 8 distribui bem o cálculo. Toda inclusão ainda é aceita.

O requisito muda a sequência de uso

```
Pedido pedido = new Pedido();  
pedido.adicionarItem(teclado, 2);  
  
// O cliente considera o pedido concluído.  
  
pedido.adicionarItem(mouse, 1);
```

O que deveria acontecer com a segunda chamada?

O comentário não muda o objeto

Com teclado a 150.0 e mouse a 80.0 :

- o código atual calcula 380.0 ;
- o requisito espera 300.0 ;
- a segunda inclusão precisa ser impedida.

Falta representar no próprio `Pedido` que ele foi fechado.

CONCEITO-CHAVE

PONTO DE PARTIDA

A mudança começa pelo estado

Um pedido passa por dois estados relevantes:

```
aberto —————> fechado  
      pedido.fechar()
```

O estado pertence a cada objeto `Pedido`.

Quem deve proteger a inclusão?

- `Main` é cliente e pode esquecer a regra;
- `ItemPedido` não conhece o estado do pedido;
- `Pedido` recebe a solicitação e mantém a coleção.

A operação que altera a coleção deve proteger a regra.

ATIVIDADE

RESPONSABILIDADE

Localize a decisão

Qual objeto deve decidir se `adicionarItem(Produto, int)` pode prosseguir?

1. `Main`, antes da chamada;
2. `ItemPedido`, antes de entrar na lista;
3. `Pedido`, dentro de `adicionarItem`.

Justifique usando o estado e as informações que cada objeto conhece.

Um campo privado representa a situação

```
private boolean fechado;
```

- `false` : o pedido está aberto;
- `true` : o pedido está fechado;
- cada `Pedido` possui seu próprio valor.

Código externo solicita ações; não altera o campo diretamente.

O pedido nasce aberto e pode fechar

```
public Pedido() {  
    itens = new ArrayList<>();  
    fechado = false;  
}  
  
public void fechar() {  
    fechado = true;  
}
```

`fechar()` expressa uma transição do domínio.

Criar o estado ainda não basta

```
pedido.fechar();  
pedido.adicionarItem(mouse, 1);
```

Se `adicionarItem` não consultar `fechado`, o item continuará entrando.

O estado precisa proteger a operação que altera a coleção.

ATIVIDADE

PROTEÇÃO DA COLEÇÃO

Em que ponto verificar?

```
ItemPedido item = new ItemPedido(produto, quantidade);  
itens.add(item);
```

A condição deve ser verificada antes ou depois dessas instruções?

O que uma tentativa recusada deve evitar?

A guarda envolve a criação e a inclusão

```
public void adicionarItem(Produto produto, int quantidade) {  
    if (!fechado) {  
        ItemPedido item = new ItemPedido(produto, quantidade);  
        itens.add(item);  
    }  
}
```

Quando `fechado` é `true`, nenhuma instrução do bloco é executada.

Conceito-chave — regra protegida pelo objeto

Depois que `Pedido` é fechado, `adicionarItem` não aumenta seu conjunto de itens.

A regra fica no objeto que conhece seu estado e controla a mudança.

A tentativa recusada preserva o estado

Nesta versão:

- `adicionarItem` mantém retorno `void` ;
- não há exceção;
- a lista continua igual;
- nenhum `ItemPedido` novo é criado.

Comunicar a recusa seria outra decisão sobre o contrato da operação.

O estado pertence a cada instância

```
Pedido primeiro = new Pedido();  
primeiro.adicionarItem(teclado, 2);  
primeiro.fechar();  
primeiro.adicionarItem(mouse, 1);
```

```
Pedido segundo = new Pedido();  
segundo.adicionarItem(mouse, 1);
```

Fechar primeiro altera segundo ?

ATIVIDADE
DOIS PEDIDOS

Preveja os dois totais

Com teclado a `150.0` e mouse a `80.0` :

1. qual total `primeiro.calcularTotal()` devolve?
2. qual total `segundo.calcularTotal()` devolve?
3. quantos `ItemPedido` são criados pelas três chamadas?

DOIS PEDIDOS

A saída mostra dois estados independentes

```
300.0  
80.0
```

- primeiro rejeita a inclusão posterior ao fechamento;
- segundo continua aberto;
- a tentativa recusada não cria um terceiro item.

O que muda no modelo?

Parte	Consequência
Pedido	guarda o fechamento e protege a inclusão
cliente	solicita fechar() e verifica o cenário
ItemPedido	continua calculando o subtotal
Produto	continua fornecendo o preço
calcularTotal()	continua delegando aos itens

ATIVIDADE

IMPACTO DA MUDANÇA

Diagnostique três propostas

1. verificar `fechado` apenas em `Main` ;
2. verificar `fechado` dentro de `Pedido.adicionarItem` ;
3. fazer `ItemPedido.calcularSubtotal()` devolver `0.0` depois do fechamento.

Qual proposta protege a inclusão? Qual altera uma responsabilidade que o requisito não pediu para mudar?

Fechar não apaga itens existentes

O requisito impede novas inclusões.

Ele não diz que:

- itens já presentes desaparecem;
- o subtotal deixa de funcionar;
- **Produto** precisa conhecer o pedido;
- toda alteração possível foi proibida.

Uma regra precisa ser tão ampla quanto o requisito que a motivou.

Uma turma pode encerrar inscrições

Uma `Turma` mantém estudantes inscritos e oferece `inscrever(estudante)` .

Depois do encerramento, novas inscrições não entram.

Quem deve proteger essa regra? O que acontece com estudantes já inscritos?

A evolução foi localizada

Pedido

- estado: aberto → fechado
- fechar()
- adicionarItem() protegido
- calcularTotal() por delegação

O requisito acrescentou uma transição e uma regra de entrada. O restante da colaboração permanece compreensível.

Para levar ao laboratório

- prever antes de modificar;
- proteger a regra no objeto responsável;
- verificar antes de criar o novo item;
- comparar pedidos diferentes;
- confirmar o que mudou e o que permaneceu igual.

No Laboratório 09, a Versão 8 evolui para um pedido que recusa novas inclusões depois do fechamento.